

Atividade: Modularização de Código e Análise Comparativa de LLMs

Premissas adotadas (ajuste conforme sua realidade): cada aluno deve usar no mínimo 2 LLMs diferentes (ex.: ChatGPT, Gemini, Claude, Copilot, DeepSeek); a entrega é um relatório único contendo as três partes descritas abaixo.

1. Objetivos de aprendizagem

Ao final da atividade, o aluno deverá ser capaz de:

- Decompor um problema em **módulos coesos** (funções com responsabilidade única).
- Distinguir **lógica de processamento** de **entrada/saída de dados**.
- Avaliar criticamente código gerado por IA quanto à sua **qualidade estrutural**, e não apenas se funciona.
- Reconhecer por que a modularização importa na prática de engenharia (manutenção, teste, reuso e trabalho em equipe).

2. O problema: Estação de Monitoramento de Grandezas

Uma estação de engenharia coleta uma **série de medições** de um sensor ao longo do tempo. O programa deve processar esse conjunto de dados e emitir um **relatório de diagnóstico**.

O sistema deve, obrigatoriamente, cumprir as seguintes etapas funcionais:

- **Receber** um conjunto de medições brutas (uma lista de valores).
- **Validar** cada medição, descartando valores fora da faixa fisicamente possível ou não numéricos.
- **Converter** a medição para a unidade de engenharia, quando aplicável.
- **Calcular estatísticas**: média, mínimo, máximo, desvio padrão e quantidade de amostras válidas vs. descartadas.
- **Classificar** cada medição válida em três estados (normal, alerta e crítico) com base em limiares.
- **Detectar eventos**: contar quantas medições ultrapassaram o limiar crítico e identificar a maior sequência consecutiva de medições acima desse limiar.
- **Gerar um relatório** formatado com todos os resultados acima.

Variante por curso

Cada aluno usa o cenário do seu curso. A estrutura do código é a mesma; mudam apenas grandeza, unidade e limiares:

Curso	Grandeza medida	Faixa válida	Limiar alerta	Limiar crítico
Telecomunicações	Potência de sinal recebido (dBm)	de -120 a 0	abaixo de -85	abaixo de -100

Curso	Grandeza medida	Faixa válida	Limiar alerta	Limiar crítico
Mecânica	Temperatura de rolamento (°C)	de -20 a 200	acima de 90	acima de 120
Elétrica	Tensão de rede (V)	de 0 a 500	desvio acima de 10% do nominal	desvio acima de 20% do nominal

Os valores acima são sugestões didáticas. O aluno pode justificar e ajustar os limiares.

3. Parte 1: Pseudo-código semi-modularizado (trabalho do aluno)

Escreva com as suas palavras (em linguagem natural ou **pseudo-código**) a solução para o problema posto. Lembre-se que sua solução será utilizada como base de entrada para as LLMs gerarem códigos modularizados em Python para resolver o problema posto.

4. Parte 2: Implementação em Python gerada por LLMs

- Escolha **no mínimo 2 LLMs** diferentes.
- Use **o mesmo prompt** para todas, garantindo comparação justa. O prompt deve descrever o problema da Seção 2. Não entregue à IA o seu pseudo-código pronto; o objetivo é ver como cada uma estrutura a solução por conta própria.
- Registre, para cada LLM: **nome e versão**, prompt exato utilizado e o código Python completo retornado.
- Cole o código de cada LLM no relatório (ou anexe os arquivos .py). Verifique que cada código **executa** com um conjunto de medições de teste fornecido por você.

Documente também se precisou de mais de uma tentativa ou de correção manual. Essa é uma informação relevante para a comparação.

5. Parte 3: Análise comparativa da modularização

Avalie cada código de LLM segundo a rubrica abaixo, atribuindo **0, 1 ou 2 pontos** por critério (0 = não atende, 1 = atende parcialmente, 2 = atende plenamente):

#	Critério de modularização	LLM A	LLM B
1	Decomposição em funções, cada uma com responsabilidade única		
2	Nomes descritivos de funções e variáveis		
3	Ausência de duplicação de código (DRY)		

#	Critério de modularização	LLM A	LLM B
4	Parâmetros e retornos bem definidos; evita variáveis globais		
5	Reaproveitamento de funções (ex.: validação usada em um só lugar)		
6	Existência de uma função principal organizando o fluxo		
7	Legibilidade geral e comentários úteis (sem excesso)		
	Total (máx. 7)		

Em seguida, escreva uma **análise crítica** (1 a 2 parágrafos) respondendo:

- Qual LLM produziu o código **mais modular** e por quê? Cite critérios da tabela.
- Em que ponto a IA **divergiu** do seu pseudo-código da Parte 1 (estruturou melhor, pior ou de forma diferente)?
- Algum código funcionava mas era **mal modularizado**? O que faltava?
- **Reflexão de engenharia:** por que a modularização importa em um sistema real de monitoramento (manutenção, teste de cada módulo, reuso entre projetos, trabalho em equipe)?

6. Entregáveis

Um relatório (PDF ou documento) contendo:

- Pseudo-código semi-modularizado autoral (Parte 1).
- Para cada LLM: identificação e código Python (Parte 2).
- Tabela comparativa preenchida, acompanhada da análise crítica (Parte 3).
- Arquivos .py executáveis de cada versão (anexos).

7. Dicas (para distribuir aos alunos)

- Funcionar não é o mesmo que estar bem escrito: um código de 80 linhas dentro de uma única função pode dar a resposta certa e ainda assim ser **mal modularizado**.
- Bons sinais de modularização: funções curtas, nomes que dizem o que fazem, dados entrando por parâmetro e saindo por return, e o print concentrado em poucos lugares.
- Use o **mesmo conjunto de medições de teste** em todas as versões para comparar com justiça.
- Ao comparar LLMs, foque na **estrutura**, não no estilo cosmético.